LangChain part 1: Lang “Chain”.

Phần 1. What is LangChain ?

1. Tổng quan về LangChain:

Hãy tưởng tượng **LLM (Large Language Model)** như ChatGPT hay GPT-4 giống như một **"bộ não trong lồng kính"**. Nó cực kỳ thông minh, biết rất nhiều thứ, nhưng lại bị cô lập. Nghĩa là các LLM không thể truy cập email của bạn, không thể đọc file PDF trong máy tính bạn, và cũng không biết sự kiện gì vừa xảy ra sáng nay.

LangChain chính là **"đôi tay" và "đôi tai"** nối bộ não đó với thế giới bên ngoài.

Về mặt kỹ thuật, LangChain là một framework mã nguồn mở giúp các lập trình viên xây dựng ứng dụng sử dụng LLM dễ dàng hơn bằng cách cung cấp các công cụ để:

- Kết nối LLM với các nguồn dữ liệu bên ngoài.
- Tương tác với môi trường (như lướt web, chạy code).
- Quản lý luồng xử lý (workflow) phức tạp.

2. Tại sao LangChain được sử dụng nhiều ?

Hiện nay có nhiều thư viện hỗ trợ AI (như LlamaIndex, Haystack), nhưng LangChain vẫn là lựa chọn số 1 vì:

- **Tính trừu tượng hóa (Abstraction):** Bạn có thể đổi từ model OpenAI sang model Google Gemini hay Anthropic chỉ bằng... một dòng code. LangChain lo phần giao tiếp bên dưới.
- **Hệ sinh thái khổng lồ:** Hỗ trợ tích hợp hơn 800 công cụ (AWS, Google, Notion, Slack, SQL...).
- **Cộng đồng mạnh:** Mọi vấn đề bạn gặp phải hầu như đều đã có người giải quyết và chia sẻ trên StackOverflow hoặc GitHub.

3. Kiến trúc cốt lõi

Để dễ hình dung, kiến trúc của LangChain hoạt động theo cơ chế **Input - Process - Output**:

1. **User Input:** Người dùng đặt câu hỏi.
2. **LangChain (The Middleman):**
    - Nhận câu hỏi.
    - Lấy thêm dữ liệu cần thiết (từ file, web, tools...).
    - Gửi tất cả (Câu hỏi + Dữ liệu) cho LLM.
3. **LLM:** Xử lý và trả về câu trả lời thô.
4. **LangChain:** Chỉnh sửa định dạng câu trả lời và gửi lại cho người dùng.

Phần 2: Một số thành phần cốt lõi của LangChain

Để làm chủ LangChain, bạn cần nắm vững 6 "viên gạch" nền tảng này. Hầu hết mọi ứng dụng AI phức tạp đều được xây dựng từ việc lắp ghép chúng lại với nhau.

1. Models: Bộ não của hệ thống

LangChain có các thư viện hỗ trợ call các model từ Google, OpenAI, Anthropic hay một số trang cung cấp API Models như OpenRouter một cách dễ dàng. Ví dụ:

```
from langchain_openai import ChatOpenAI
import os
llm = ChatOpenAI(
    model="openai/gpt-oss-20b:free",
    openai_api_base="https://openrouter.ai/api/v1",
    openai_api_key=os.environ["OPENROUTER_API_KEY"],
)
```

2. Prompts: Khuôn mẫu câu lệnh

Thay vì gõ cứng (hard-code) câu lệnh như `"Hãy dịch câu 'Hello' sang tiếng Việt"`, chúng ta dùng **PromptTemplates** để tạo ra các khuôn mẫu tái sử dụng được.

Ví dụ, bạn tạo một khuôn mẫu dịch thuật:

```
from langchain_core.prompts import PromptTemplate

template = PromptTemplate.from_template("Hãy dịch từ '{word}' sang
tiếng {language}.")

prompt = template.format(word="Love", language="Pháp")

# Kết quả: "Hãy dịch từ 'Love' sang tiếng Pháp."
```

3. Output Parser:

Kết quả trả về từ Model thường là một đối tượng phức tạp (như AIMessage) hoặc văn bản thô. Output Parsers giúp chuyển đổi kết quả này thành định dạng mong muốn (chuỗi văn bản sạch, JSON, List, v.v.) để code của bạn dễ dàng xử lý tiếp.
Ví dụ: Dùng StrOutputParser để lấy ra nội dung văn bản thuần túy:

```
from langchain_core.output_parsers import StrOutputParser

parser = StrOutputParser()
# Giả sử 'response' là kết quả từ bước trên (AIMessage)
clean_text = parser.invoke(response)
# Kết quả: "Thủ đô của Việt Nam là Hà Nội." (Dạng string, không còn là
object)
```

4. Chains (Chuỗi) & LCEL: "Keo dính" kết nối

Đây là sức mạnh thực sự của LangChain. Thay vì gọi rời rạc từng bước, bạn kết nối chúng lại thành một dây chuyền (Chain). Hiện nay, LangChain sử dụng **LCEL (LangChain Expression Language)** với cú pháp | (pipe) rất trực quan, giống như trong Unix.

Ví dụ: Kết nối Prompt -> Model -> Parser thành một chuỗi hoàn chỉnh:

```
# Kết hợp 3 thành phần: Prompt | Model | Output Parser

chain = prompt | llm | parser #đều lấy từ các thành phần từ ba ví dụ
kia.

# Chạy cả chuỗi chỉ với 1 lệnh

result = chain.invoke({"word": "Love", "language": "Pháp"})

# Luồng đi:

# 1. Prompt tạo ra: "Hãy dịch từ 'Love' sang tiếng Pháp."

# 2. Model xử lý và trả về AIMessage.

# 3. Parser chuyển thành string.

# Kết quả cuối cùng: "Amour"
```

5. Retrieval (Truy xuất dữ liệu): Bộ nhớ ngoài:

LLM bị giới hạn bởi dữ liệu huấn luyện. Để AI trả lời dựa trên dữ liệu riêng của bạn (file PDF, web, database), chúng ta dùng Retrieval. Thành phần này thường bao gồm:

- **Document Loaders:** Đọc dữ liệu.
- **Text Splitters:** Chia nhỏ văn bản.
- **Vector Stores:** Lưu trữ dữ liệu dưới dạng số (embedding) để tìm kiếm.

Ví dụ: Giả lập việc tìm kiếm thông tin liên quan (Retrieving):

```
from langchain_community.vectorstores import FAISS

from langchain_openai import OpenAIEmbeddings

# Giả sử bạn có 1 danh sách văn bản

texts = ["LangChain là framework cho LLM.", "Python là ngôn ngữ lập
trình phổ biến."]

vectorstore = FAISS.from_texts(texts, embedding=OpenAIEmbeddings())

# Tạo retriever để tìm kiếm

retriever = vectorstore.as_retriever()

docs = retriever.invoke("LangChain là gì?")

# Kết quả: Tìm thấy văn bản "LangChain là framework cho LLM."
```

6. (Preview) Agents & Graphs – Tư duy có điều kiện

Phần này sẽ được nói kỹ ở Part sau, nhưng cần hiểu trước:

- **Chain**: Luồng thẳng
- **Agent**: Luồng có quyết định
- **Graph**: Luồng có nhánh, vòng lặp, state

Đây là nơi LangChain vượt xa "prompt + LLM".

Phần 3: Kết luận:

Qua Part 1, ta có thể rút ra một điểm cốt lõi:

LLM bản thân chỉ là một bộ não xử lý ngôn ngữ.
LangChain chính là hệ thần kinh nối bộ não đó với thế giới thực.

Thông qua Part 1, chúng ta đã:

- Thấy được vai trò của LangChain như một *middle layer* điều phối
- Nắm được các **thành phần nền tảng**: Model, Prompt, Output Parser, Chain, Retrieval
- Hiểu tư duy **Input → Process → Output** khi xây dựng ứng dụng LLM

Điều quan trọng cần nhấn mạnh là:

> **LangChain không phải là một thư viện để "viết prompt cho đẹp", mà là một framework để thiết kế hệ thống AI có cấu trúc, có kiểm soát và có khả năng mở rộng.**